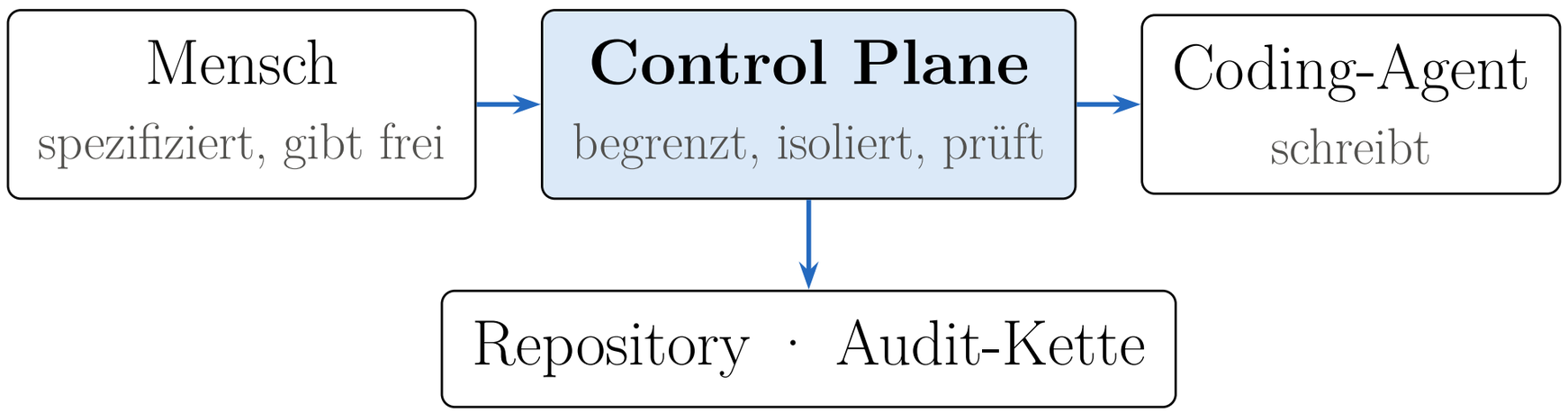
Der Engpass ist nicht das Modell. Es ist die fehlende Prozessschicht.

Coding-Agenten rufen Werkzeuge selbstständig auf. Das macht sie produktiv — und ihre Ausführung nichtdeterministisch und sicherheitsrelevant. Befunde der überarbeiteten Fassung, Stand Oktober 2026.

Nicht möglichst viele Agenten. Eine Steuerschicht dazwischen.



Die Control Plane sitzt zwischen Mensch, Coding-Agent und Zielrepository. Sie begrenzt, isoliert, koordiniert, prüft und macht nachweisbar.

Ein Schreiber je Arbeitskopie. Mehrere unabhängige Prüfer.

Zwei Agenten, die gleichzeitig dieselbe Arbeitskopie verändern, erzeugen Schreibkonkurrenz, unklare Zurechnung und Zwischenstände, für die niemand mehr geradestehen kann.

Wer schreibt, gibt nicht frei.

Erzeugung und Freigabe sind technisch getrennt. Read-only-Reviewer liefern strukturierte Befunde — und ein Agent bewertet nie allein die eigene Arbeit.

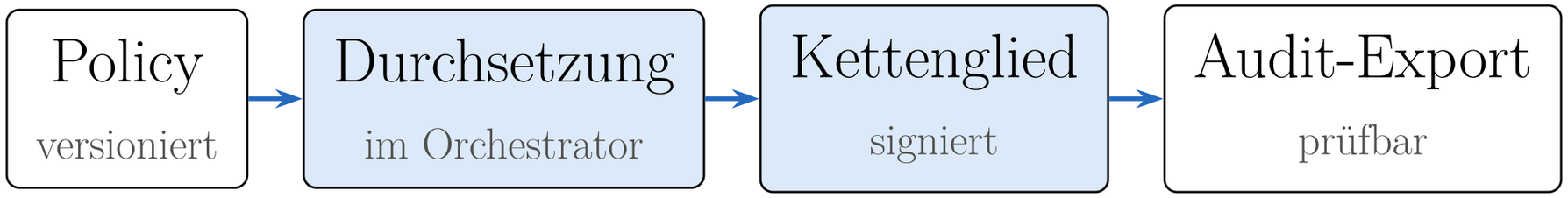
Ein Gate, dessen Ausfall wie Erfolg aussieht, ist schlimmer als kein Gate.

Ein ausgefallener Pflicht-Reviewer, eine nicht prüfbare Policy, eine fehlgeschlagene Attestierung: Nichts davon darf jemals als bestanden durchgehen.

Git trägt die Artefakte. Die Datenbank trägt den Prozesszustand.

Branches und Checkpoints gehören in Git. Workflow-, Policy-, Freigabe- und Audit-Zustand liegen autoritativ in der Datenbank — sonst ist der Prozess nicht rekonstruierbar.

Für jeden Lauf rekonstruierbar: welche Policy, welches Modell, welche Freigabe.



Jede Durchsetzung erzeugt ein signiertes Kettenglied in einer append-only Audit-Hashkette. Governance ist damit keine Ergänzung der Architektur, sondern Teil ihrer Struktur.

Vom Konzept zum System: 34 fachliche Slices, 47 Releases in sechs Monaten.

Rund 55.000 Zeilen Produktivcode, Testabdeckung als buildbrechendes Gate (Line ab 85 %, Branch ab 81 %). Die Architektur des Whitepapers war die Grundlage — das Kapitel beschreibt, was daraus geworden ist. Stand: 20. Oktober 2026, Release 0.39.0.

Parallelität entsteht aus Abhängigkeiten — nicht aus Agentenrollen.

Umgesetzt bis Release 0.30.0, hinter einem standardmäßig deaktivierten Feature-Flag. Der Weg ging bewusst stufenweise: erst parallele Analyse ohne Schreibrechte, dann die Schreibrechte — gebunden an eine Besitzregel —, dann versionierte Verträge, dann begründungspflichtige Planänderungen, dann Merge-Konflikte als Diagnose statt Sammelkategorie, zuletzt Koordinationsschicht und eine Messung, die ihre Lücken ausweist. Der verteilte Betrieb bleibt zurückgestellt, solange der Bedarf nicht gemessen ist. Seit 0.35.0 arbeiten die Läufe auf Wunsch gegen freigegebene Specs mit nummerierten Akzeptanzkriterien, seit 0.39.0 auch per Policy erzwungen.

Whitepaper

Agentic Software Development

Enterprise Architecture with AI Agents

Von kontrollierten Einzel-Runs zur parallelen Agentic Software Factory. Praxisbeispiele mit Java, Spring Boot und Domain-Driven Design

Vollständiges Whitepaper: Download-Link im Beitrag

janda.io · Überarbeitete und erweiterte Fassung 3.2, Stand: Oktober 2026.